ElectroAdvisor AI — Intelligent Electronics Sales Assistant

> **AI Engineer Technical Assessment** · 3-Day Build Challenge · Electronics Recommendation Agent

ElectroAdvisor is a production-grade AI-powered sales assistant designed for an electronics store. Customers often know their practical goals (e.g., college coursework, local machine learning experiments, carrying between campus lectures), but rarely know the exact technical specifications they need.

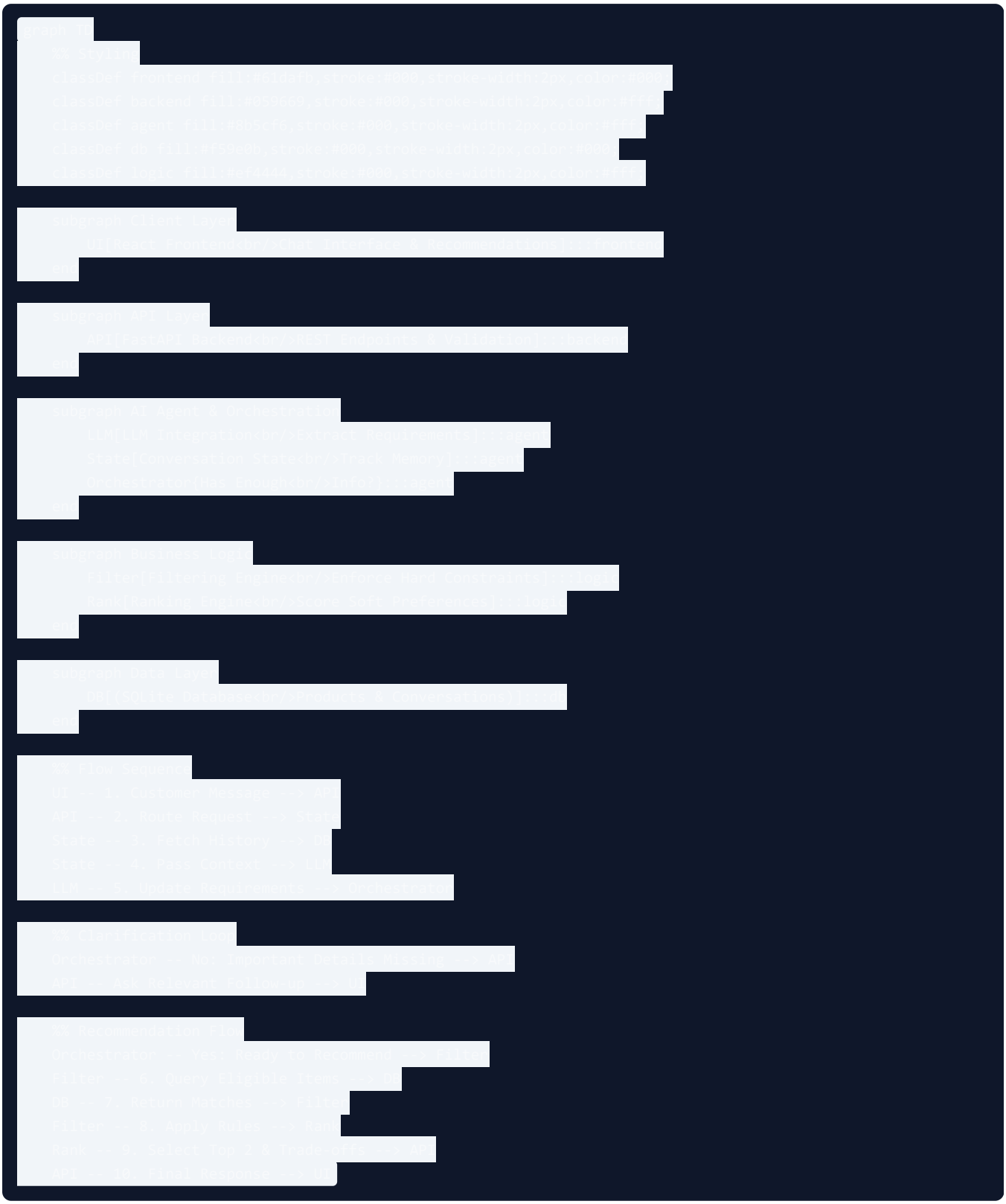
ElectroAdvisor engages in a natural multi-turn conversation, dynamically tracks customer requirements across messages, differentiates **hard constraints** (e.g., maximum budget ceiling) from **soft preferences** (e.g., weight, portability, battery life), uses **controlled database query tools**, and recommends the **two most suitable products** grounded in real catalog data with explicit reasons and trade-offs.

📑 Table of Contents

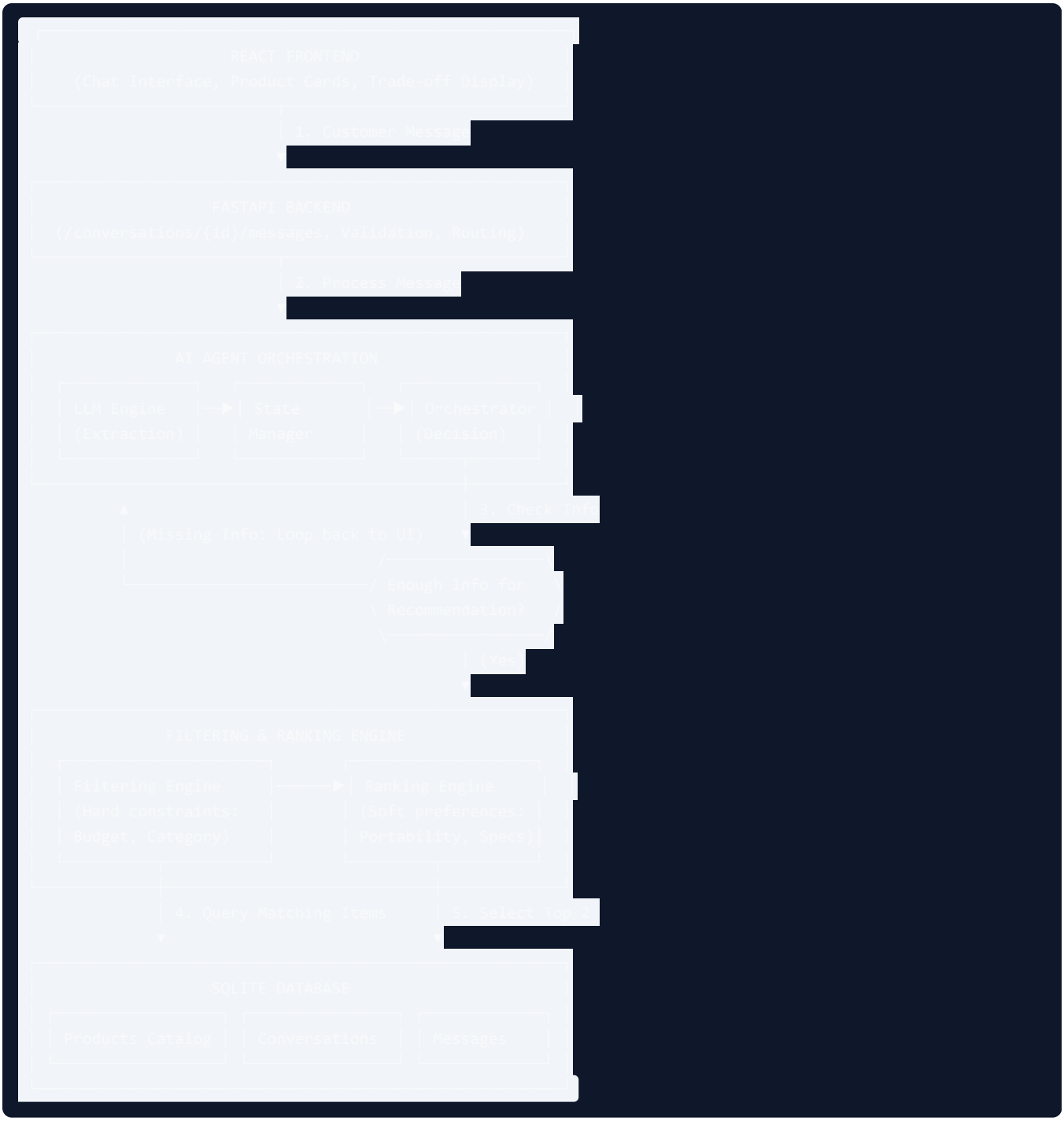
1. Architecture & Decision Flow
2. FARM Stack Compliance
3. Filtering & Ranking Engine Specification
4. Project Directory Structure
5. Quickstart Setup Guide
6. Automated Test Suite
7. Complete 12-Endpoint API Architecture
8. Edge Cases & Reliability
9. Assessment Timeline & Check-ins

🏛️ Architecture & Decision Flow

Mermaid.js Architecture Flowchart



ASCII Box Architecture Diagram



FARM Stack Compliance

The assessment requires **FastAPI** plus at least one more component from the **FARM** stack (FastAPI, React, MongoDB).

- **FastAPI:** Core backend REST API with asynchronous endpoints, Pydantic v2 schemas, and structured logging.
- **React:** Modern single-page application built with Vite and responsive Vanilla CSS, featuring live requirement memory inspection, product recommendation cards, and a comparative trade-off matrix.
- **Relational Data Layer:** SQLite via SQLAlchemy 2.0 ORM with synthetic catalog seeding.

🔍 Filtering & Ranking Engine Specification

Stage 1: Hard Constraints Enforcement (Application Logic)

Mandatory constraints are strictly enforced in application code before ranking:

1. **Category Filter:** Product category must match target category (e.g., `laptop`, `smartphone`).
2. **Strict Maximum Budget Cap:** Any product where `product.price > budget_max` is immediately disqualified.
3. **Hardware Mandatory Thresholds:**
    - If customer requires **local machine learning**, application logic requires `min_ram_gb >= 16` and a **dedicated NVIDIA RTX GPU** with CUDA tensor cores (or Apple Unified Memory).
    - If customer requires **gaming**, dedicated GPU is mandatory.
4. **Stock Availability:** `product.stock > 0`.

> **Honest No-Match Handling:** If 0 products satisfy mandatory constraints (e.g. an RTX 4090 laptop under ₹30,000), the system does **not** hallucinate products or violate constraints. Instead, it explains the exact bottleneck and suggests realistic alternatives.

Stage 2: Soft Preference & Use-Case Ranking Algorithm

Products passing Stage 1 are scored on a scale from 0 to 100 points:

$$\text{Score} = S_{\text{use-case}} + S_{\text{portability}} + S_{\text{battery}} + S_{\text{value}} + S_{\text{rating}}$$

Component	Max Points	Evaluation Logic
Use-Case Alignment ($S_{\text{use-case}}$)	40 pts	Matches between customer's `primary_uses` and `product.intended_uses`. Bonus for dedicated VRAM $\ge$ 6GB on ML tasks.
Portability ($S_{\text{portability}}$)	20 pts	Weight $\le$ 1.35kg (+20 pts), 1.35–1.65kg (+15 pts), 1.65–2.15kg (+8 pts), > 2.3kg (+2 pts). Amplified when campus commute is mentioned.
Battery Life (S_{battery})	15 pts	Runtime ≥ 14h (+15 pts), ≥ 9h (+12 pts), ≥ 6.5h (+8 pts), < 6.5h (+4 pts).
Budget Optimization (S_{value})	15 pts	Evaluates hardware efficiency within budget without exceeding the ceiling.
Rating & Reliability (S_{rating})	10 pts	$(\text{Rating} / 5.0) \times 10$.

Stage 3: Complementary Top-2 Selection

- **Rank #1:** Highest overall scoring model.
- **Rank #2:** Highest scoring model that offers a distinct, meaningful trade-off (e.g., maximum GPU compute vs. lightweight campus portability & battery endurance).

📁 Project Directory Structure



🚀 Quickstart Setup Guide

Prerequisites

- **Python 3.10+** (Tested on Python 3.12)
- **Node.js 18+** & **npm**

1. Backend Setup

```
# Navigate to the backend directory
cd backend

# Create and activate virtual environment
python -m venv venv

# Windows PowerShell
.\venv\Scripts\Activate.ps1
# Linux / macOS
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Initialize and seed the SQLite database with 25 synthetic products
python -m app.data.init_db

# Start the FastAPI server (running on http://127.0.0.1:8000)
uvicorn app.main:app --reload --port 8000
```

> **API Documentation:** Interactive Swagger UI is available at **http://127.0.0.1:8000/docs**.

2. Frontend Setup

Open a second terminal window:

```
# Navigate to the frontend directory
cd frontend

# Install Node dependencies
npm install

# Start the Vite development server (running on http://127.0.0.1:5173)
npm run dev
```

Open **http://127.0.0.1:5173** in your browser.

🧪 Automated Test Suite

The backend includes a comprehensive automated test suite built with `pytest` covering 20 distinct test cases across functional requirements, edge cases, and the assessment benchmark conversation:

```
cd backend

# Windows
.\venv\Scripts\python.exe -m pytest

# Linux / macOS (or with active virtual environment)
pytest
# or
python -m pytest backend/tests
```

Docker Quickstart (Optional)

You can run the entire FARM stack (FastAPI backend + React frontend) with a single command:

```
docker-compose up --build
```

- Frontend available at: `http://localhost:5173`
- Backend API available at: `http://localhost:8000` (Swagger docs at `http://localhost:8000/docs`)

🧠 Why This LLM / Agent Approach Fits the Task

Section 4 of the assessment invites candidates to choose an LLM / agent framework and explain why the approach fits the problem.

We chose an **Orchestrated Stateful Agent with a Dual-Mode (LLM + Deterministic NLP) Extractor**:

1. **Zero Hallucination of Database Records:** Naive LLM agents that generate SQL or choose products directly frequently hallucinate model numbers, prices, or technical specifications. In ElectroAdvisor, the LLM is restricted to **structured entity extraction** (turning text into Pydantic models). Product retrieval, filtering, and scoring are executed strictly in deterministic Python code and SQLAlchemy ORM.
2. **Explicit Decision Orchestrator vs. Blind Questionnaires:** Instead of relying on a rigid state machine questionnaire or an unconstrained LLM loop, an explicit Decision Graph evaluates requirement completeness. It asks a focused question when essential fields (use case, budget, or local vs. cloud) are missing, and transitions immediately to recommendation once requirements are sufficient.
3. **Reproducible, Zero-Cost Evaluation:** By supporting Google Gemini and OpenAI with a deterministic fallback, recruiters and automated CI pipelines can clone and test the repository without needing paid API keys or hitting external rate limits.

📡 Complete 12-Endpoint API Architecture

Based on architecture review and engineering rigor, the API specification has been expanded from the baseline 6 endpoints to a **complete 12-endpoint architecture** to support hybrid UI controls, debuggability, side-by-side comparison, taxonomy browsing, and user feedback loops.

Category	Method	Endpoint	Purpose & Engineering Justification
Catalog	`GET`	`/api/v1/products`	List, filter by price/brand/search, and paginate catalog items deterministically
Catalog	`GET`	`/api/v1/products/{product_id}`	Retrieve comprehensive specs, intended uses, pros, and cons for a specific item
Catalog	`GET`	`/api/v1/products/compare`	**UX Enhancement:** Side-by-side specification matrix & trade-offs (`?ids=LAP-001,LAP-004`)
Catalog	`GET`	`/api/v1/categories`	**UI Optimization:** Distinct categories, item counts, price ranges, and brands for UI chips
Agent	`POST`	`/api/v1/conversations`	Initialize a new stateful customer sales consultation session
Agent	`POST`	`/api/v1/conversations/{id}/messages`	Process customer natural language input and execute decision graph
Agent	`GET`	`/api/v1/conversations/{id}`	Retrieve full message history, extracted requirements, and memory state
Agent	`PUT`	`/api/v1/conversations/{id}/requirements`	**Hybrid UI Control:** Direct state override via sliders/chips without typing in chat
Agent	`DELETE`	`/api/v1/conversations/{id}`	**Privacy & Reset:** Wipe conversation messages and requirements state safely
Agent	`POST`	`/api/v1/conversations/{id}/feedback`	**Product Analytics:** Record user sentiment (thumbs up/down) & purchase conversion intent
System	`GET`	`/api/v1/health`	Service uptime and database connectivity health check
System	`GET`	`/api/v1/conversations/{id}/debug`	**Engineering Rigor:** Inspect active constraints, decision traces, and candidate scoring

---

Endpoint Payloads & Examples

1. `GET /api/v1/categories`

Response (200 OK):



2. `GET /api/v1/products/compare?ids=LAP-001,LAP-004`

Response (200 OK):



3. `PUT /api/v1/conversations/{id}/requirements`

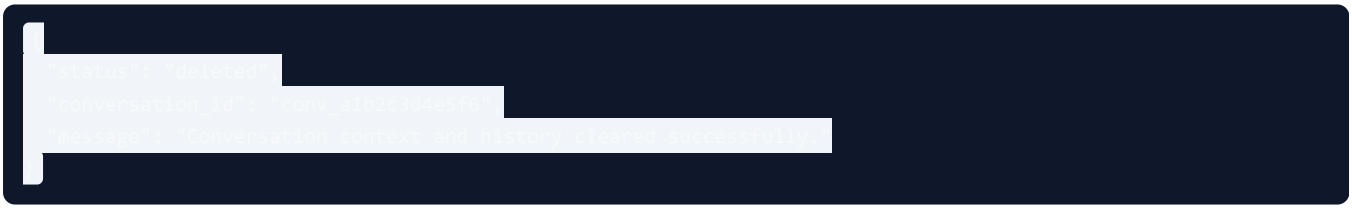
Request Body:



Response (200 OK): Returns updated `ConversationResponse` with synchronized state.

4. `DELETE /api/v1/conversations/{id}`

Response (200 OK):



5. `POST /api/v1/conversations/{id}/feedback`

Request Body:



Response (200 OK): Status `received` with timestamp and saved feedback metadata.

6. `GET /api/v1/conversations/{id}/debug`

Response (200 OK):



🛡 Edge Cases & Reliability

1. **Controlled Tool Execution:** Queries use parameterized SQLAlchemy ORM filters. The agent cannot execute arbitrary SQL.
2. **Indian Number Format Support:** Correctly parses Indian currency groupings (e.g. `₹1,20,000` parses to `120000.0`, `1.5 lakh` parses to `150000.0`).
3. **Contradictory Constraints Detection:** Detects contradictions (e.g. `min_price > max_price` or "MacBook running Windows") and asks for clarification.
4. **Category Isolation:** Switching categories (e.g., from laptop to monitor) resets hardware constraints so laptop RAM requirements do not bleed into monitor queries.
5. **Post-Recommendation Follow-ups:** Understands user refinements ("any lighter options?", "thanks") and responds appropriately without repeating the initial recommendation.
6. **Graceful LLM Fallbacks:** Validates LLM responses with Pydantic schemas, imposes a 5-second timeout, and falls back to deterministic rules if the LLM output is malformed.

🗓️ Assessment Timeline & Check-ins

Day 1 Check-in (19 October 2026, 9:00 PM): Architecture Skeleton

- **Status: Completed & Submitted**
- **Artifacts:** Documented in docs/architecture.md and viewable in the React web UI. Includes Mermaid.js flowchart and ASCII box diagram.

Day 2 Check-in (20 October 2026, 9:00 PM): Progress Report

- **Architecture Evolution:** Based on architecture review, the API specification has been expanded from the baseline 6 endpoints to 12 endpoints to support hybrid UI controls, debuggability, and user feedback loops.
- **What is Implemented So Far:**
  - ✅ **FastAPI Backend REST API (Complete 12 Endpoints):** All 12 endpoints implemented with Pydantic validation, deterministic sorting, and error handling.
  - ✅ **Two-Stage Recommendation Engine:** Application-level hard filtering (budget cap, category, RAM, dedicated GPU) followed by soft preference utility scoring (0–100 pts).
  - ✅ **State Tracking & Orchestrator:** Stateful conversation memory across turns, category isolation, and semantic clarification tracking.
  - ✅ **Dual-Mode Extractor:** Support for Gemini and OpenAI with Pydantic validation, 5s timeout, and regex rule engine.
  - ✅ **React Frontend:** Chat UI with live State Inspector, product recommendation cards, and trade-off comparison table.
  - ✅ **Automated Tests:** 37 automated tests passing with pytest across all 12 endpoints and core edge cases.
- **What Works:**
  - Multi-turn benchmark conversation from Section 3 (College $\rightarrow$ Programming & ML $\rightarrow$ Around 80k & campus carry $\rightarrow$ Top 2 recommendations strictly within ₹80k).
  - Mind changes (updating budget mid-conversation).
  - Category transitions (laptop to monitor/headphones).
  - Indian currency parsing (`₹1,20,000` , `80k`).
  - Dynamic database-computed no-match explanations.
  - Side-by-side product comparisons (`/api/v1/products/compare`).
  - Taxonomy loading for chips/dropdowns (`/api/v1/categories`).
  - UI state override (`PUT /api/v1/conversations/{id}/requirements`).
  - Session wipe and privacy reset (`DELETE /api/v1/conversations/{id}`).
  - Recommendation feedback collection (`POST /api/v1/conversations/{id}/feedback`).
  - Engineering diagnostics and candidate traces (`GET /api/v1/conversations/{id}/debug`).
- **Current Limitations & Known Areas for Improvement:**
  - *Headless Browser Driver in Sandbox:* Automated Playwright browser subagents encountered an external CDN mirror 404 issue downloading Windows driver binaries; local manual browser testing works normally.
  - *Catalog Depth:* The synthetic catalog has 23 items with depth concentrated in laptops (14 models) and 2–3 items in other categories.
  - *Rule Engine Primary Usage:* In standard evaluation without an external API key, extraction relies on pattern matching; while robust for common queries, complex phrasing benefits from providing a `GEMINI_API_KEY` .
- **What Remains for Day 3:**
  - Expanding the product catalog to 35+ items for richer non-laptop recommendations.
  - Live demonstration walkthrough rehearsal for the Google Meet.

Day 3 Check-in (21 October 2026, Google Meet): Final Submission & Demo

- **Status:** Ready for live demonstration.
- **Artifacts:** Source code, automated tests, final README, and interview defense guide in docs/demo_guide.md.